SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA4303

COURSE OUTCOME COVERED

CO-1: Develop well-structured, standards-compliant web pages using HTML/XHTML and XML, and apply Internet protocols and HTTP communication mechanisms for web content delivery. (L2)

Assessment Tool: Application-Based Questions

Weightage: 40%

QUESTIONS: **Total Marks: 30**

Code of Conduct:

*I **Nisha K (192411231)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:
Nisha K

RUBRICS FOR CALCULATION:

Criteria	Weig htage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconception s	Poor understandin g; major misconceptio ns
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand

1: Online Symposium Portal

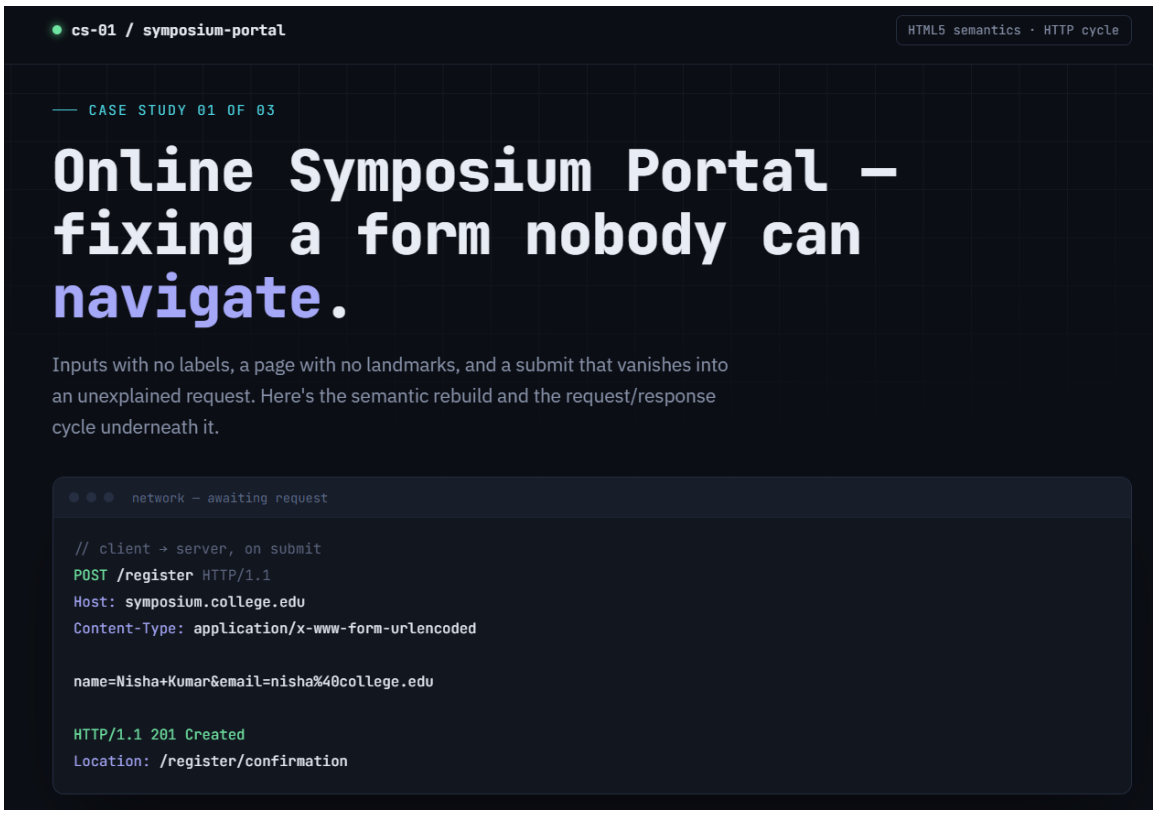
A college is developing an online symposium registration portal. The following HTML structure and HTTP interaction is observed:

```
<form action="/register" method="POST">
  <input type="text" name="name">
  <input type="email" name="email">
  <button>Submit</button>
</form>
```

When a student submits the form, the browser sends an HTTP request to the server, and the server responds accordingly.

Questions:

1. Identify appropriate **HTML5 semantic elements** missing in the structure.
2. Modify the structure to make it **standards-compliant**.
3. Interpret the **HTTP request generated** during form submission.
4. Analyze the **HTTP response cycle** from server to browser.
5. Suggest improvements for **usability and accessibility**.



Q1–Q2 — What's missing, and the fix

Q1 — MISSING SEMANTIC ELEMENTS

What the raw markup leaves out

- `<label>` for every input — nothing for a screen reader to announce
- `<main>` , `<header>` , `<section>` — no page landmarks
- `<fieldset>` / `<legend>` — no grouping or purpose for the fields
- explicit `type="submit"` , `required` , matched `id` / `for` pairs

Q5 — USABILITY & ACCESSIBILITY

What the fix adds

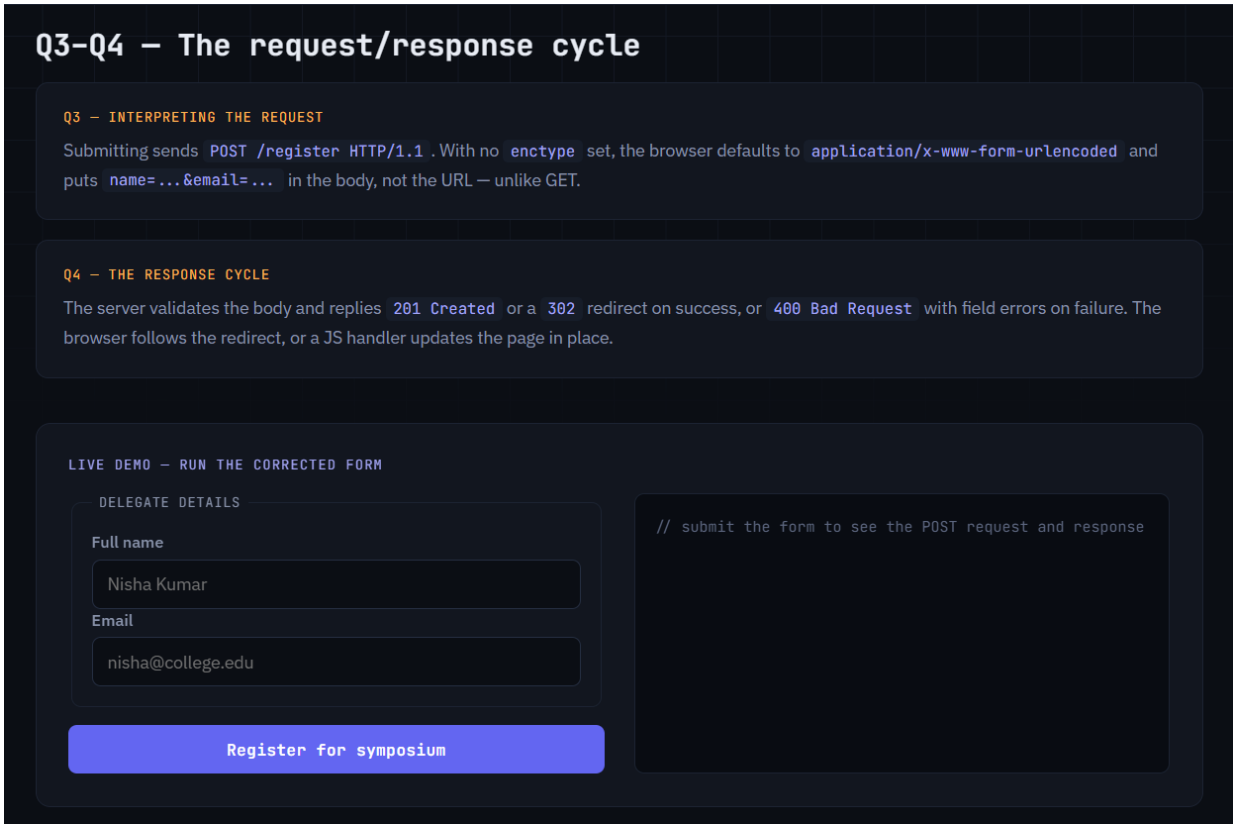
- Visible `<label>` s, not placeholders (placeholders vanish on input)
- Inline errors tied via `aria-describedby`
- Native validation (`required` , `type="email"`) before any JS layer
- A visible focus ring and an `aria-live` status region

BEFORE

```
<form action="/register" method="POST">
  <input type="text" name="name">
  <input type="email" name="email">
  <button>Submit</button>
</form>
```

STANDARDS-COMPLIANT

```
<main>
 <form action="/register" method="POST">
  <fieldset>
   <legend>Delegate details</legend>
   <label for="name">Full name</label>
   <input id="name" name="name" required>
   <label for="email">Email</label>
   <input id="email" name="email"
    type="email" required>
  </fieldset>
  <button type="submit">Register</button>
 </form>
</main>
```


Q3–Q4 — The request/response cycle
Q3 — INTERPRETING THE REQUEST
Submitting sends POST /register HTTP/1.1 . With no enctype set, the browser defaults to application/x-www-form-urlencoded and puts name=...&email=... in the body, not the URL — unlike GET.
Q4 — THE RESPONSE CYCLE
The server validates the body and replies 201 Created or a 302 redirect on success, or 400 Bad Request with field errors on failure. The browser follows the redirect, or a JS handler updates the page in place.
LIVE DEMO — RUN THE CORRECTED FORM
DELEGATE DETAILS
Full name
Nisha Kumar
Email
nisha@college.edu
Register for symposium
// submit the form to see the POST request and response

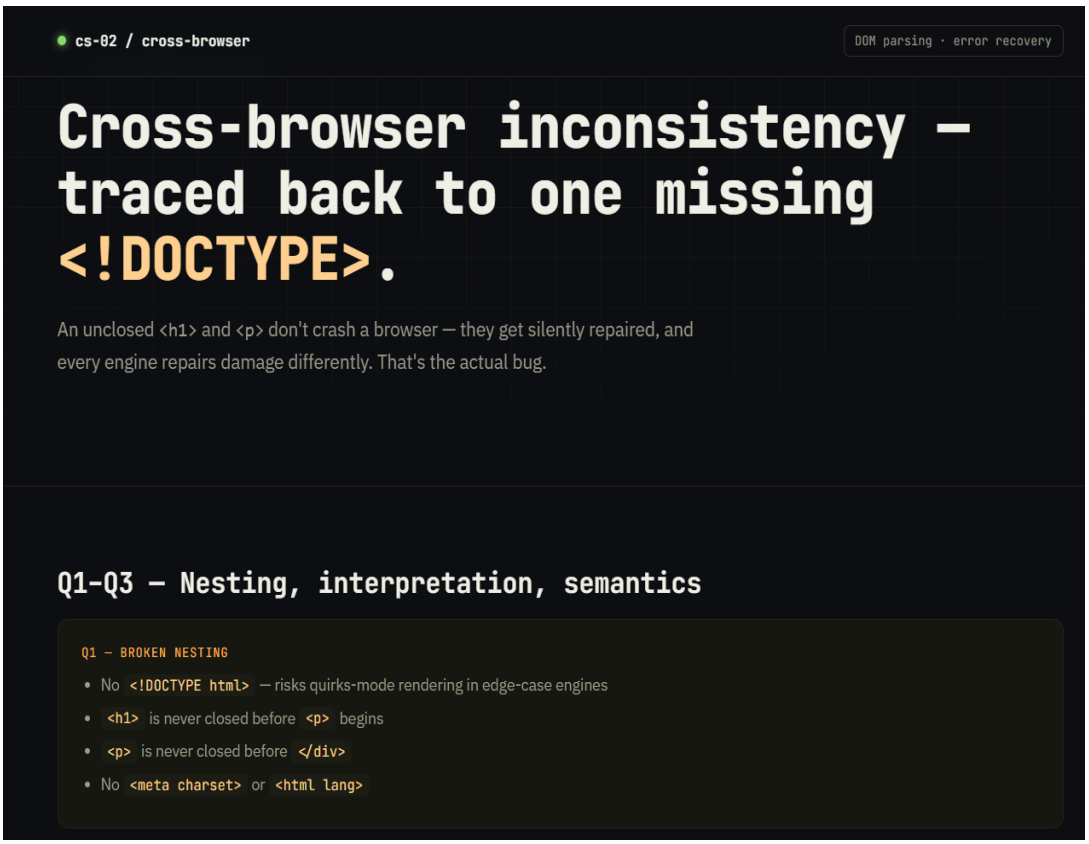
2: Cross-Browser Compatibility Issue

A company website shows inconsistent layout across browsers. The following HTML snippet is used:

```
<html>
  <head>
    <title>Company</title>
  </head>
  <body>
    <div>
      <h1>Welcome
      <p>About us
    </div>
  </body>
</html>
```

Questions:

1. Identify **improper nesting and missing closing tags**.
2. Analyze how different browsers may interpret this structure.
3. Identify **missing semantic elements** (e.g., header, section).
4. Provide a **corrected W3C-compliant HTML structure**.
5. Suggest techniques to ensure **cross-browser compatibility**.



Q2 — HOW ENGINES INTERPRET IT

HTML parsers use error-recovery rules, not strict rejection. Since `<p>` is a block element, most engines implicitly close the open `<h1>` — but which ancestor gets auto-closed, and how the DOM ends up nested, is left to each engine's own recovery heuristics.

Q3 — MISSING SEMANTIC ELEMENTS

A bare `<div>` wrapping a heading and paragraph carries no meaning. It should be a `<header>` for the intro and a `<section>` for the "about us" copy, so both assistive tech and CSS can target them reliably.

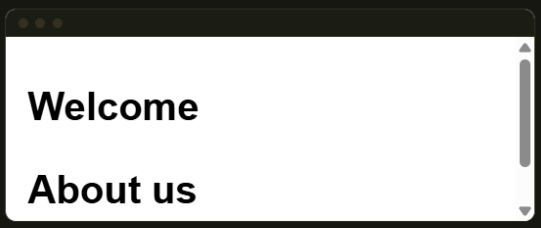
Q5 — CROSS-BROWSER TECHNIQUES

- Always declare `<!DOCTYPE html>` to force standards mode
- Validate markup against the W3C validator before shipping
- Use a CSS reset/normalize layer so box models agree
- Feature-detect with `@supports` instead of browser-sniffing

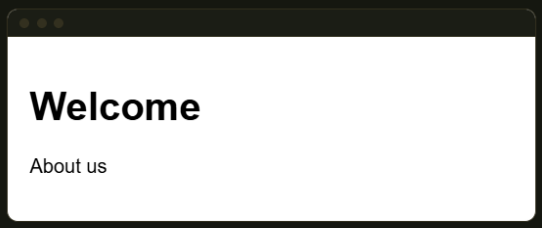
Q4 — Same DOM, rendered both ways

LIVE DEMO — HOVER EITHER FRAME

BROKEN MARKUP, AUTO-REPAIRED HERE



CORRECTED, VALID NESTING



Both look similar here because this engine recovers gracefully — that's the trap: it hides the ambiguity a stricter or older parser would expose as a real layout break.

CAUSE	EFFECT WITHOUT DOCTYPE / VALID NESTING
Missing <!DOCTYPE>	Quirks mode: different box-model & sizing rules per engine
Unclosed <h1>	Engine-specific auto-close point → inconsistent DOM depth
No landmark elements	Screen readers and CSS selectors have nothing reliable to target
Corrected markup	Standards mode, identical DOM tree across engines

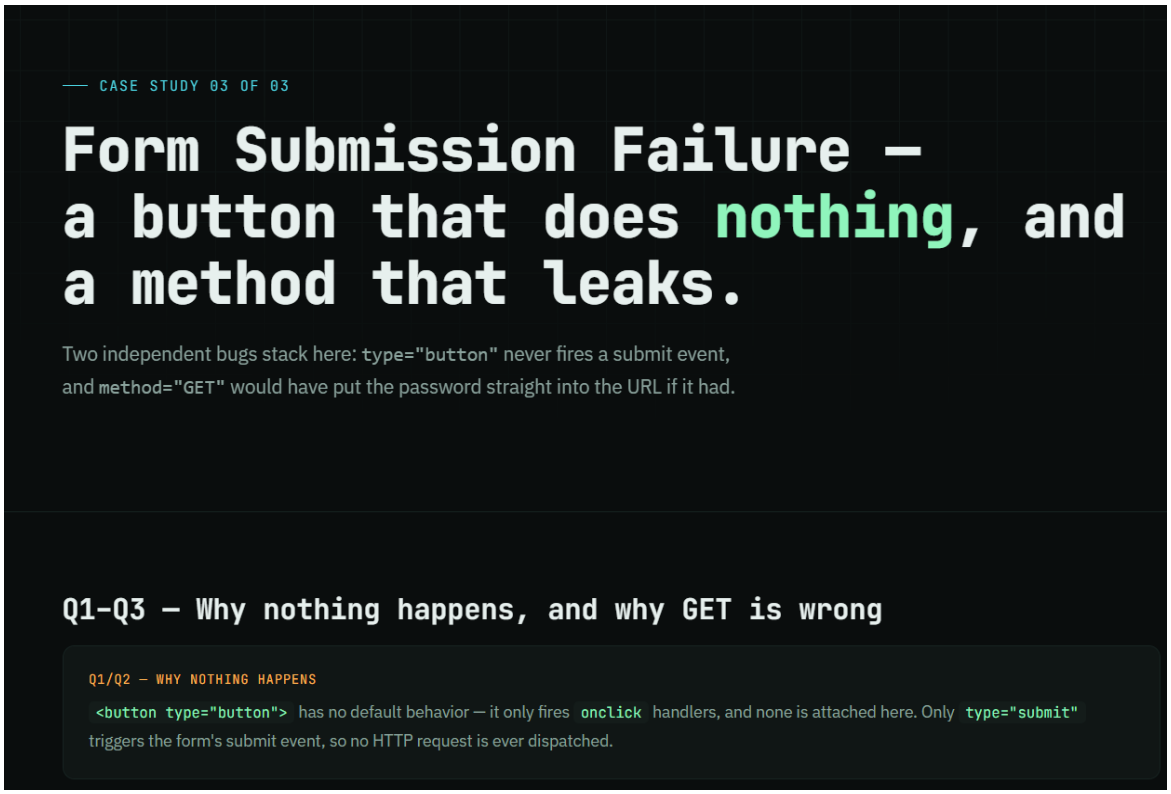
3: Form Submission Failure

A web application fails to send form data to the server. The following configuration is observed:

```
<form action="/submit" method="GET">
  <input type="text" name="username">
  <input type="password" name="password">
  <button type="button">Login</button>
</form>
```

Questions:

1. Identify issues in the **form configuration**.
2. Analyze why the **HTTP request is not triggered**.
3. Interpret the role of **GET method in this scenario**.
4. Propose a **corrected implementation**.
5. Suggest improvements for **secure data transmission**



Q4-Q5 — Corrected implementation & secure transmission

LIVE DEMO — BROKEN VS. FIXED, CLICK BOTH

ORIGINAL — TYPE="BUTTON"

Username

never reaches the server

Password

••••••••

Login

no submit event | GET would expose password

CORRECTED — POST + TYPE="SUBMIT"

Username

nisha

Password

••••

Login

fires on submit | body-encoded, not URL

```
POST /submit HTTP/1.1
Host: portal.college.edu
Content-Type: application/x-www-form-urlencoded

username=nisha&password=•••••••• (body, not URL)

HTTP/1.1 200 OK
Set-Cookie: session=…; HttpOnly; Secure; SameSite=Strict
```

Q3 — THE ROLE OF GET

`GET` serializes every field into the query string: `/submit?username=x&password=y`. That string lands in browser history, server access logs, and any proxy in between — plaintext, bookmarkable, and shareable. Wrong for any credential-bearing form.

BEFORE — NEVER SUBMITS, UNSAFE IF IT DID

```
<form action="/submit" method="GET">
  <input type="text" name="username">
  <input type="password" name="password">
  <button type="button">Login</button>
</form>
```

AFTER — FIRES, CREDENTIALS STAY IN THE BODY

```
<form action="/submit" method="POST">
  <label for="u">Username</label>
  <input id="u" name="username" required>
  <label for="p">Password</label>
  <input id="p" name="password"
   type="password" required>
  <button type="submit">Login</button>
</form>
```

PROPERTY	GET (BEFORE)	POST (AFTER)
Where data travels	Query string, in the URL	Request body
Visible in browser history	Yes	No
Visible in server access logs	Often, yes	Not by default
Suitable for credentials	Never	Yes, over HTTPS

Q5 — SECURE TRANSMISSION CHECKLIST

- ✓ Serve the page over **HTTPS** so POST bodies are TLS-encrypted in transit
- ✓ Hash + salt passwords server-side (bcrypt/argon2) — never log or store them raw
- ✓ Add a CSRF token and rate-limit the login endpoint
- ✓ Set `autocomplete="current-password"` and cookies `HttpOnly; Secure; SameSite=Strict`